STAFF REVIEW · H446 010 source models

Separate from the eight-page pupil workbook. This copy is in a public repository and is NOT access controlled. Check technical correctness before use.

Printed question mapping: Booklet Q1 = original online Q1; Booklet Q2 = original online Q4

Do Now — source answers

1. = assigns a value. == compares two values for equality.

2. Unicode supports a much larger range of characters and writing systems.

3. 5.5 Mantissa 01011000 = 0.6875 ; exponent 0011 = 3 ; $0.6875 \times 2^3 = 5.5$.

4. Distinction. The first condition is true, so its branch runs and the later branches are skipped. Conditions are tested top to bottom. The first true branch runs, so the later branch is skipped.

I Do — source worked steps

1. Read the number — CODE:
Console.Write("Number: ");
int number =
int.Parse(Console.ReadLine()); — This happens once before the repetition begins.

2. Count from 1 to 5 — CODE:
for (int i = 1; i <= 5; i++)
{
// repeated instruction
} — i = 1 starts the counter; i <= 5 keeps the loop running; i++ adds 1 after each repetition.

3. Calculate each multiple — CODE:
for (int i = 1; i <= 5; i++)
{
int answer = number * i;
Console.WriteLine(answer);
} — The same two instructions run five times, but i changes each time.

4. Trace a known input — CODE:
number = 7

i = 1 → 7
i = 2 → 14
i = 3 → 21
i = 4 → 28
i = 5 → 35

We Do — guided CHECK

1. Complete the pattern
CODE:
for (int i = 1; i <= 5; i++)
{

```
Console.WriteLine(i * i);
}
```

2. What controls the loop?

3. Reverse the OCR stopping condition
A do...while runs the body before checking the condition.

4. Describe this loop
CODE:
```
while (number > 0)
{
number = number / 2;
}
```
A precise answer names the construct, the repeated code and the continuation condition.

You Do — source mark schemes

1. Countdown
CODE:
```
using System;

public class Program
{
public static void Main()
{
for (int i = 10; i >= 1; i--)
{
Console.WriteLine(i);
}
}
}
```
Why it works: the counter starts at 10, continues while it is at least 1, and decreases by 1 after each repetition.

2. Times-table generator
CODE:
```
using System;

public class Program
{
public static void Main()
{
Console.Write("Enter a whole number: ");
int number = int.Parse(Console.ReadLine());

for (int i = 1; i <= 10; i++)
{
int answer = number * i;
Console.WriteLine(answer);
```

```
}
}
}
```
Why it works: the loop runs exactly ten times and the counter provides the multiplier.

3. Validated age
CODE:
```
using System;

public class Program
{
public static void Main()
{
bool valid = false;
int age = 0;

while (valid == false)
{
Console.Write("Enter an age from 0 to 120: ");
age = int.Parse(Console.ReadLine());

if (age >= 0 && age <= 120)
{
valid = true;
}
else
{
Console.WriteLine("Invalid - try again.");
}
}

Console.WriteLine("Accepted");
}
}
```
Key idea: the Boolean remains false until the accepted condition is met.

4. Password attempts
CODE:
```
using System;

public class Program
{
public static void Main()
{
string answer;
int attempts = 0;

do
```

```
{
Console.Write("Password: ");
answer = Console.ReadLine();
attempts++;
}
while (answer != "computer");

Console.WriteLine("Attempts: " + attempts);
}
}
```

Key idea: do...while is suitable because the password must be asked at least once. The counter increases on every attempt.

Exit Ticket — source answer

1. False. A while loop checks its condition before each iteration and may run zero times.